Promises in JavaScript

The Beginner's Guide to Async JS, Fetch, and Await

1. The Problem: Waiting in JavaScript

Imagine you are a chef in a kitchen. You are the **only** chef (JavaScript is "single-threaded"). You get an order to bake a cake, which takes 30 minutes in the oven.

If you just stand in front of the oven doing nothing for 30 minutes, the rest of the restaurant freezes. No one gets their food. Instead, you put the cake in the oven, set a timer, and go chop vegetables for the next order. When the timer rings, you go back to the cake.

This is exactly how **Asynchronous JavaScript** works. It allows JS to start a long task (like fetching data from the internet), move on to other code, and come back when the task finishes.

2. What is a Promise? (The Restaurant Buzzer)

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation.

Think of it like going to a busy restaurant. The host hands you a glowing buzzer. That buzzer is a "Promise." It doesn't give you a table instantly, but it guarantees that eventually, you will either get a table, or they will tell you they are closed.

- **Pending:** You are holding the buzzer. The data is still loading.
- **Fulfilled (Resolved):** The buzzer flashes! The data arrived successfully.
- **Rejected:** The network failed or the server crashed.

> 💡 **Tutor's Fun Fact**
>
> In the old days of JavaScript, Promises were not built into the language! Developers had to download third-party libraries like `Bluebird` or `Q` just to use them. Today, Promises are natively baked right into JavaScript.

3. Building & Consuming a Promise (Steps 1 & 2)

In the video, Hitesh writes 5 distinct promises to show how they work step-by-step. To build a promise, we use `new Promise`. It gives us two buttons we can press: `resolve` (success) and `reject` (failure).

```
// PROMISE 1: Creating it const promiseOne = new Promise(function(resolve, reject) { // Simulating a network request that takes 1 second setTimeout(function() { console.log("Async task is complete"); resolve(); // We press the success button to connect it to .then() }, 1000); }); // Consuming it (Waiting for it to finish) promiseOne.then(function() { console.log("Promise consumed!"); });
```

4. Passing Data (Promise 3)

The host doesn't just buzz you; they tell you your table number. When we call `resolve()`, we can pass data (like an object or an array) straight into it. Whatever we pass is automatically caught by the `.then()`.

```
const promiseThree = new Promise(function(resolve, reject) { setTimeout(function() { // Passing an object full of data directly into resolve resolve({ username: "Chai", email: "chai@example.com" }); }, 1000); }); // The 'user' variable automatically catches the object passed by resolve promiseThree.then(function(user) { console.log(user.username); // Prints: "Chai" });
```

5. Handling Errors: Reject & Catch (Promise 4)

What if the network fails? We use the `reject()` button. To handle a rejection gracefully, we attach a `.catch()` block to the end of our chain.

```
const promiseFour = new Promise(function(resolve, reject) {
setTimeout(function() { let error = true; // Let's pretend a network error
happened if (!error) { resolve({ username: "hitesh", password: "123" }); } else {
reject("ERROR: Something went wrong"); // Pressing the fail button } }, 1000);
}); // The Chaining Magic promiseFour .then((user) => { console.log(user); return
user.username; // Passes data to the NEXT .then() }) .then((username) => {
console.log(username); // Prints just the username }) .catch((err) => { // This
catches our "ERROR: Something went wrong" message console.log(err); })
.finally(() => { // This runs ALWAYS, no matter if it resolved or rejected
console.log("The promise is either resolved or rejected"); });
```

💡 **What is .finally()?**

`.finally()` is the cleanup crew. Imagine you show a loading spinner on your website while fetching data. Whether the data succeeds (`then`) or fails (`catch`), you need to hide the loading spinner at the very end. You do that inside `.finally()` .

6. The Modern Upgrade: async and await (Promise 5)

Writing `.then().then().catch()` over and over can get messy. In 2017, JavaScript gave us a cleaner way to write the exact same thing: `async` and `await` .

- **await:** Think of this as a "pause button". It tells JavaScript to pause execution on this exact line until the Promise finishes.
- **async:** You are strictly forbidden from using the `await` pause button unless you label the surrounding function with the word `async` .

```
const promiseFive = new Promise(function(resolve, reject) {
setTimeout(function() { let error = true; if (!error) { resolve({ username:
"javascript", password: "123" }); } else { reject("ERROR: JS went wrong"); } },
1000); }); // Using async/await async function consumePromiseFive() { try { // We
press "pause" and wait for the promise to finish const response = await
promiseFive; console.log(response); } catch (error) { // If the promise hits
reject(), it jumps straight here! console.log(error); } } consumePromiseFive();
```

⚠️ Why do we need a try...catch block?

This is a massive trap for beginners! When you use `.then()`, you have `.catch()` to handle errors gracefully. But `await` does **not** have a built-in catch. If the promise gets rejected and you don't use a `try...catch` block, the `await` line will violently crash your entire program.

7. Enter `fetch()`: The Ultimate Promise

In the real world, you rarely write `new Promise` yourself. Usually, you just want to get data from a URL. JavaScript gives us a built-in tool called `fetch()`.

`fetch()` is basically a robot that goes to a URL, grabs the data, and comes back. Because this takes time, **fetch automatically returns a Promise!**

```
async function getAllUsers() { try { // 1. Send the fetch robot (returns a Promise) const response = await fetch('https://api.github.com/users/hiteshchoudhary'); // 2. The internet sends pure Text. We MUST translate it to JSON! // Translating also takes time, so it requires its own 'await' const data = await response.json(); console.log(data); } catch (error) { console.log("E: ", error); } } getAllUsers();
```

Doing the exact same fetch using .then()

```
fetch('https://api.github.com/users/hiteshchoudhary') .then((response) => { // Translate text to JSON (returns a promise) return response.json(); }) .then((data) => { // Log the translated data console.log(data); }) .catch((error) => console.log(error));
```

🔥 Huge Interview Question: Fetch and the 404 Error

If you use `fetch()` to go to a URL that does not exist (resulting in a 404 Not Found error), does that trigger the `.catch()` block or the `.then()` block?

Answer: It triggers the `.then()` block! `fetch()` only ever rejects (goes to catch) if there is a *physical network failure* (like your Wi-Fi dropping). A 404 error means the browser successfully reached the server, and the server successfully replied with "I can't find that." Since communication happened, it resolves as a success!